

实验五：鸿蒙多端应用开发

实验项目基本信息

- 实验编号：**d20301035105、d20301009205
- 学时分配：**4学时
- 实验类型：**验证型
- 每组人数：**6人
- 对应课程目标：**课程目标3

一、实验概述

本实验旨在开发一个基于HarmonyOS的“智慧天气”应用，实现天气信息展示、多端自适应布局、传感器数据采集以及分布式数据同步功能。通过本实验，掌握ArkUI声明式UI开发、响应式布局设计、传感器API调用和分布式数据管理等核心技术。

1.1 核心步骤

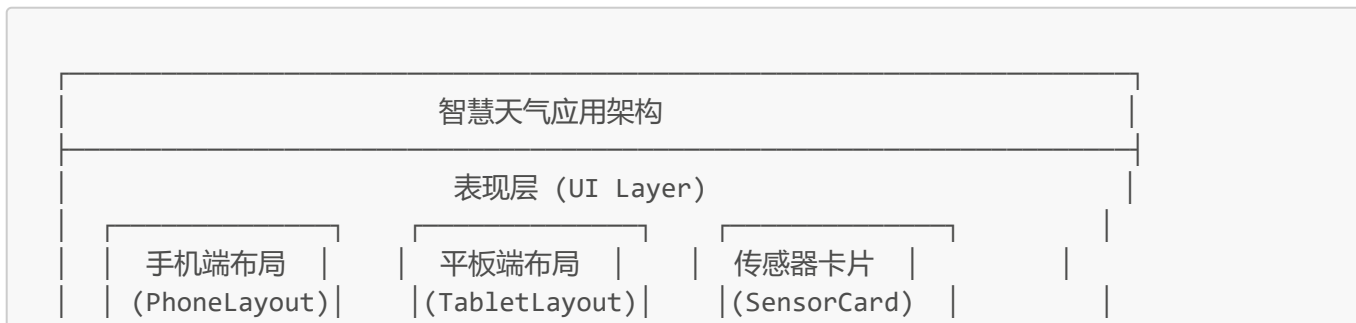
阶段	主要内容	关键技术点
需求分析	定义功能需求与多端适配策略	需求建模
AI辅助编码	生成ArkUI页面布局与传感器调用代码	TRAE辅助开发
测试验证	在不同设备模拟器上验证适配效果	多设备测试
技术报告	编写多端适配与硬件集成分析报告	技术文档撰写

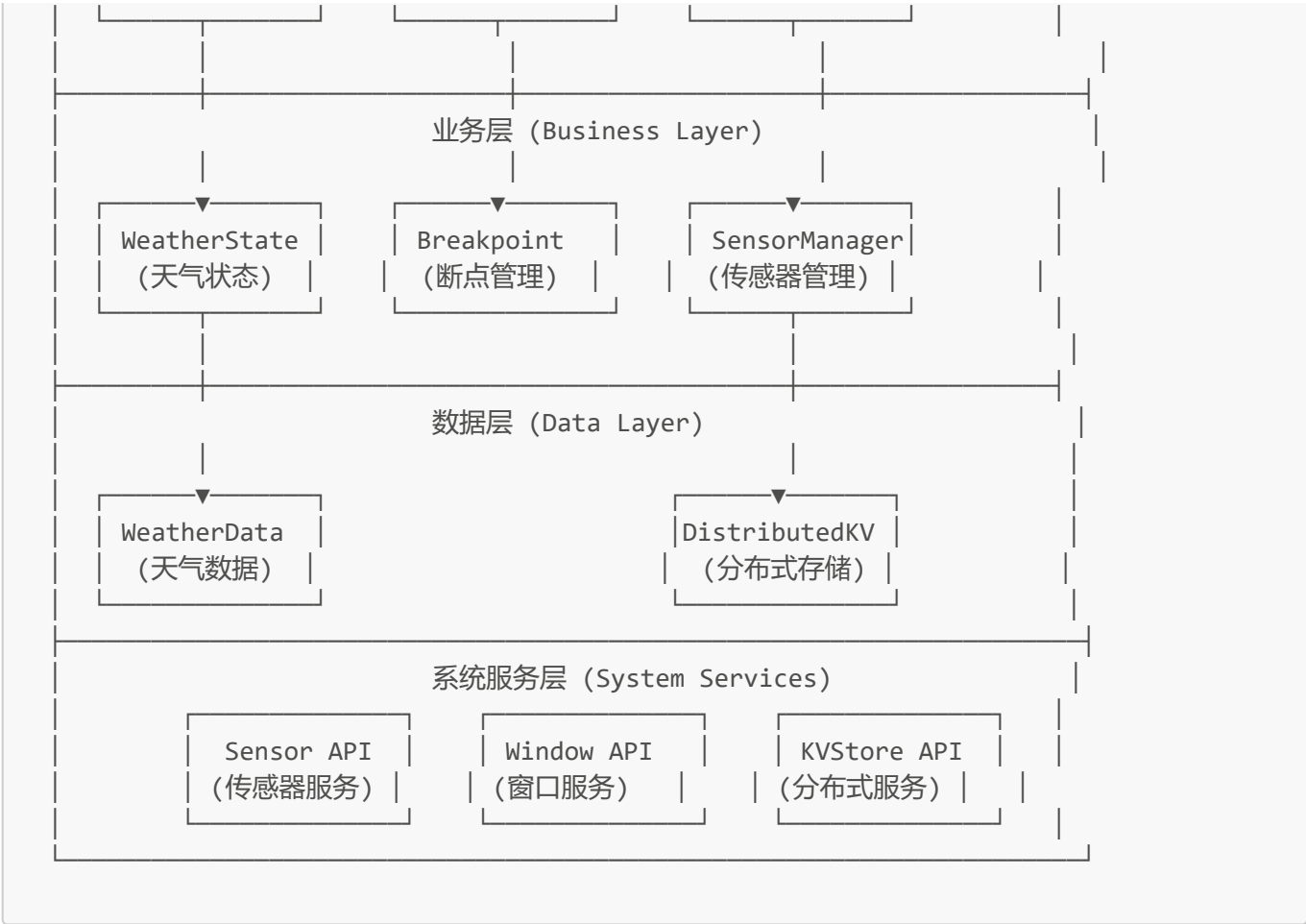
1.2 成功标准

- ☒ 天气应用功能完整
- ☒ 手机/平板界面自适应
- ☒ 传感器数据正常采集
- ☒ 多端适配效果良好
- ☒ 技术分析报告完成

二、系统架构设计

2.1 整体框架图





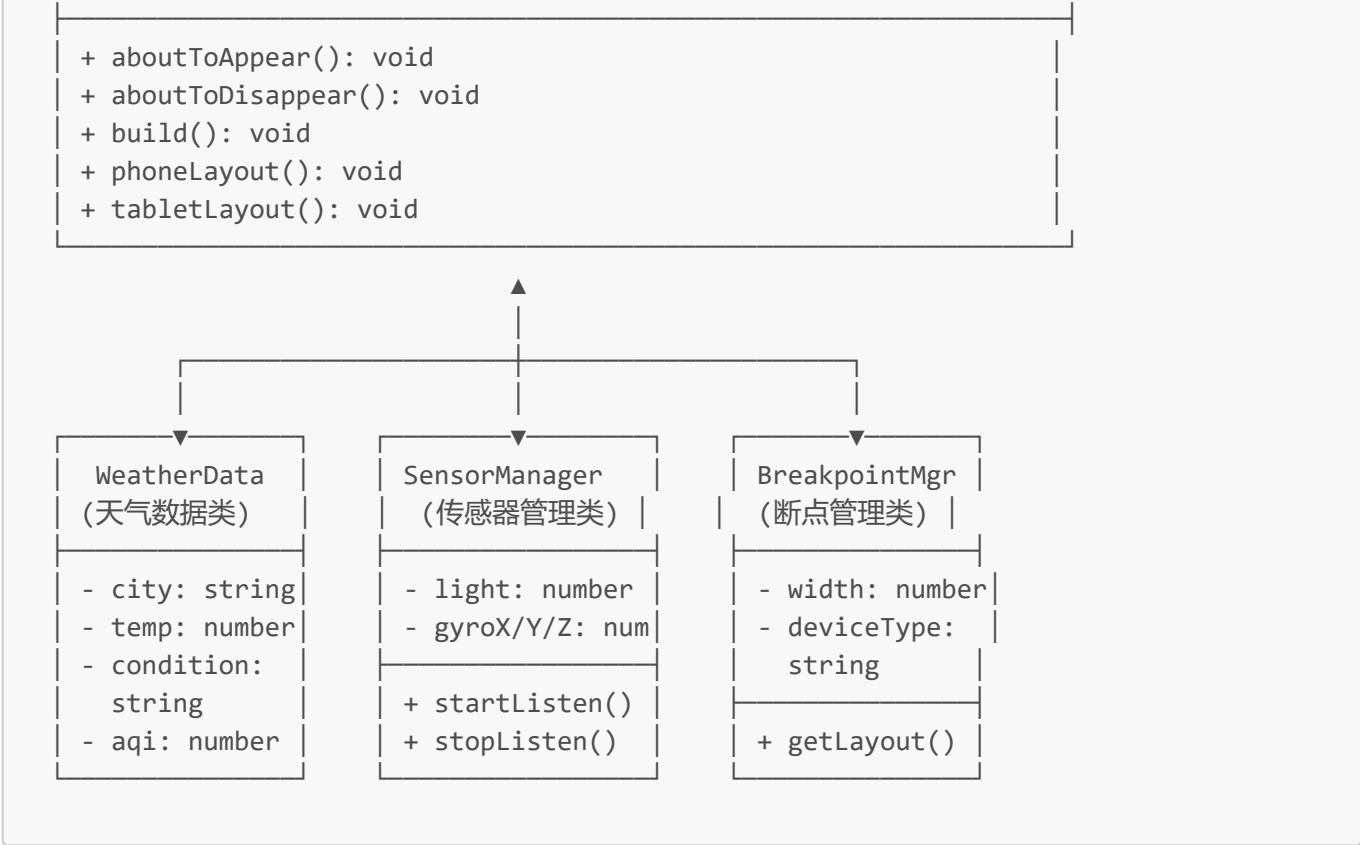
2.2 架构层次说明

层级	职责	核心组件
表现层	负责UI展示与用户交互	PhoneLayout、TabletLayout、SensorCard
业务层	管理应用状态与业务逻辑	WeatherState、Breakpoint、SensorManager
数据层	数据存储与管理	WeatherData、DistributedKV
系统服务层	提供系统能力支持	Sensor API、Window API、KVStore API

三、核心类设计

3.1 类结构图





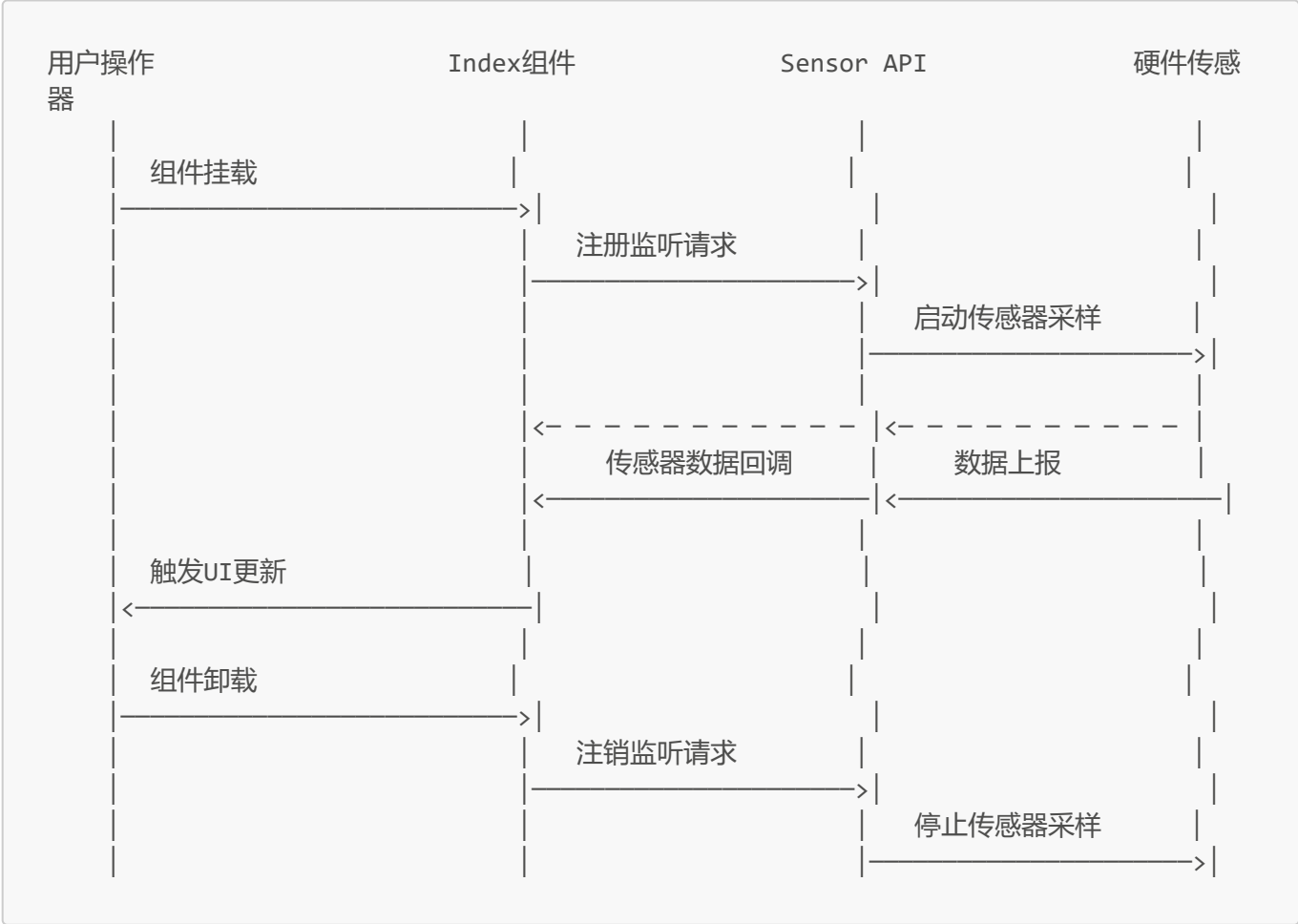
属性	类型	说明
light	number	光线传感器数据
gyroX/Y/Z	number	陀螺仪三轴角速度
方法	说明	
startListen()	注册传感器监听器	
stopListen()	注销传感器监听器	

3.2.3 BreakpointMgr 断点管理类

属性	类型	说明
width	number	当前窗口宽度
deviceType	string	设备类型(phone/tablet)
方法	说明	
getLayout()	根据窗口宽度返回布局类型	

四、执行流程分析

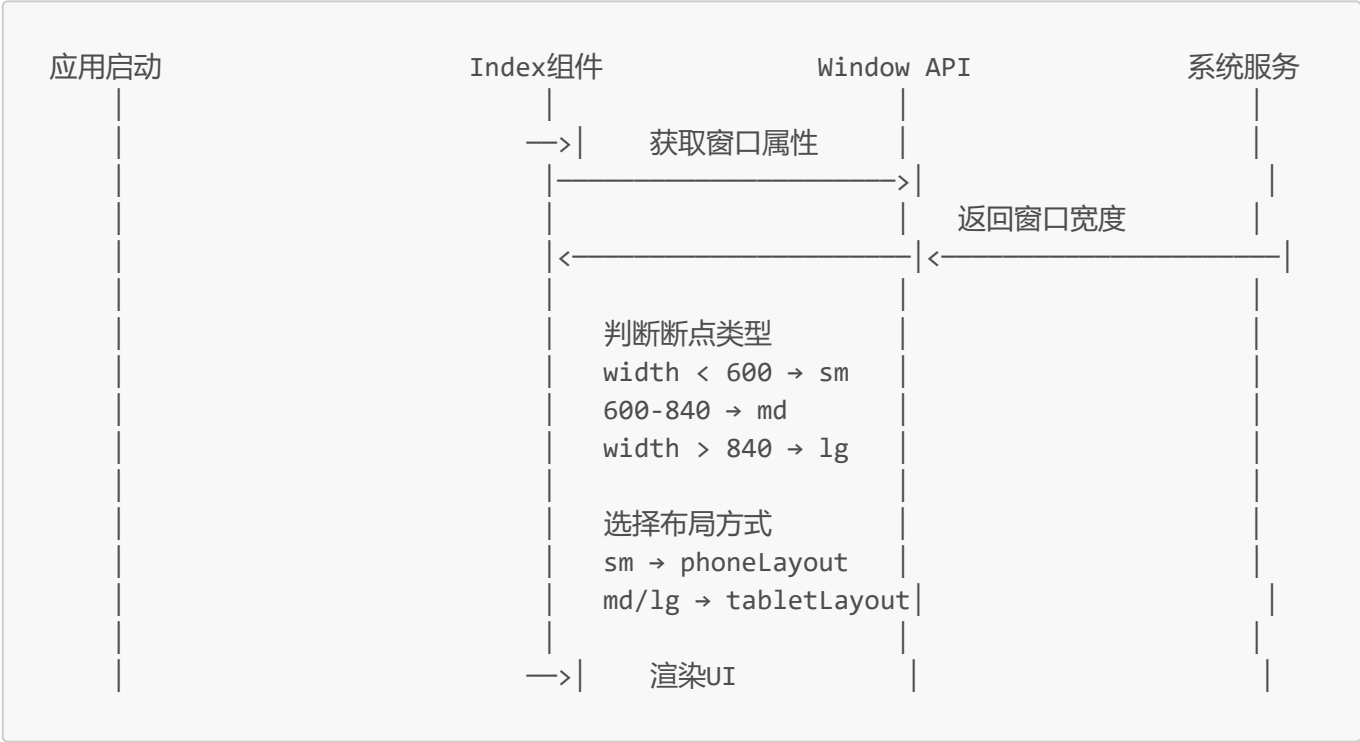
4.1 传感器数据采集顺序图



4.2 流程说明

步骤	参与者	动作	说明
1	用户	组件挂载	页面初始化
2	Index	注册监听	调用sensor.on()
3	Sensor API	启动采样	向硬件发送启动指令
4	硬件传感器	数据上报	周期性返回传感器数据
5	Sensor API	数据回调	通过callback返回数据
6	Index	UI更新	@State状态驱动界面刷新
7	用户	组件卸载	页面销毁
8	Index	注销监听	调用sensor.off()
9	Sensor API	停止采样	释放硬件资源

4.3 多端适配流程



五、核心代码实现

5.1 天气首页主组件

```
// ets/pages/index/Index.ets
@Entry
@Component
struct Index {
  @State cityName: string = '合肥';
```

```
@State temperature: number = 22;
@State weather: string = '晴';
@State aqi: number = 45;
@State lightIntensity: number = 0;
@State rotateX: number = 0;
@State rotateY: number = 0;
@State rotateZ: number = 0;
@State currentBreakpoint: string = 'md';

aboutToAppear() {
    // 获取窗口宽度判断设备类型
    let windowHeight = window.getWindowProperties().windowWidth;
    this.currentBreakpoint = windowHeight < 600 ? 'sm' : (windowWidth < 840 ? 'md'
: 'lg');

    // 注册光线传感器
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {
        this.lightIntensity = data.light;
    });

    // 注册陀螺仪传感器
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
        this.rotateX = data.x;
        this.rotateY = data.y;
        this.rotateZ = data.z;
    });
}

aboutToDisappear() {
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE);
}

build() {
    Column() {
        if (this.currentBreakpoint === 'sm') {
            this.phoneLayout()
        } else {
            this.tabletLayout()
        }
    }
    .width('100%')
    .height('100%')
    .backgroundColor('#4A90D9')
}

@Builder phoneLayout() {
    Column({ space: 10 }) {

Text(this.cityName).fontSize(28).fontColor('#fff').fontWeight(FontWeight.Bold)
        Text(`${this.temperature}
°C`).fontSize(72).fontColor('#fff').fontWeight(FontWeight.Lighter)
        Text(this.weather).fontSize(24).fontColor('#fff')
        Text(`空气质量: ${this.aqi} - 优`).fontSize(16).fontColor('#00ff00')
```

```

// 传感器数据卡片
Column({ space: 10 }) {
  Text('光线传感器').fontSize(18).fontColor('#fff')
  Text(`光线强度: ${this.lightIntensity}
lux`).fontSize(16).fontColor('#fff')

  Text('陀螺仪').fontSize(18).fontColor('#fff').margin({ top: 20 })
  Row({ space: 30 }) {
    Text(`X: ${this.rotateX.toFixed(2)}`).fontSize(14).fontColor('#fff')
    Text(`Y: ${this.rotateY.toFixed(2)}`).fontSize(14).fontColor('#fff')
    Text(`Z: ${this.rotateZ.toFixed(2)}`).fontSize(14).fontColor('#fff')
  }
}
.padding(20)
.backgroundColor('rgba(255,255,255,0.2)')
.borderRadius(12)
.margin({ top: 30 })
}
.padding(30)
.width('100%')
}

@Builder tabletLayout() {
  Row({ space: 80 }) {
    Column({ space: 15 }) {

Text(this.cityName).fontSize(40).fontColor('#fff').fontWeight(FontWeight.Bold)
      Text(`${this.temperature}
°C`).fontSize(96).fontColor('#fff').fontWeight(FontWeight.Lighter)
      Text(this.weather).fontSize(32).fontColor('#fff')
      Text(`空气质量: ${this.aqi} - 优`).fontSize(20).fontColor('#00ff00')
    }

    Column({ space: 20 }) {
      Column({ space: 10 }) {
        Text('光线传感器').fontSize(24).fontColor('#fff')
        Text(`光线强度: ${this.lightIntensity}
lux`).fontSize(20).fontColor('#fff')
      }
      .padding(25)
      .backgroundColor('rgba(255,255,255,0.2)')
      .borderRadius(12)

      Column({ space: 10 }) {
        Text('陀螺仪').fontSize(24).fontColor('#fff')
        Row({ space: 50 }) {
          Text(`X轴
\n${this.rotateX.toFixed(2)}`).fontSize(18).fontColor('#fff').textAlign(TextAlign.
Center)
          Text(`Y轴
\n${this.rotateY.toFixed(2)}`).fontSize(18).fontColor('#fff').textAlign(TextAlign.
Center)
          Text(`Z轴

```

```
\n${this.rotateZ.toFixed(2)}}`).fontSize(18).fontColor('#fff').textAlign(TextAlign.Center)\n    }\n  }\n  .padding(25)\n  .backgroundColor('rgba(255,255,255,0.2)')\n  .borderRadius(12)\n}\n}\n.width('100%')\n.height('100%')\n.justifyContent(FlexAlign.Center)\n.padding(40)\n}\n}
```

六、测试结果展示

6.1 手机端运行结果





说明：手机端采用纵向布局，天气信息垂直排列，传感器数据以卡片形式展示在底部。界面适配手机竖屏显示，字体大小和间距针对小屏幕进行了优化。

6.2 平板端运行结果



说明：平板端采用横向布局，左侧展示天气主信息，右侧展示传感器数据卡片。界面充分利用平板的宽屏优势，布局更加开阔，字体更大更清晰。

6.3 测试验证记录

测试项	测试结果	说明
天气信息展示	通过	城市、温度、天气状况正常显示
多端适配	通过	手机/平板布局自动切换
光线传感器	通过	数据实时更新，单位显示正确
陀螺仪传感器	通过	三轴数据正常采集
UI美观度	通过	配色协调，布局合理

七、技术分析报告

7.1 多端适配技术分析

断点系统设计：

- **sm (手机)**：窗口宽度 < 600px，采用单列垂直布局
- **md (平板竖屏)**：600px ≤ 宽度 < 840px，采用双栏布局
- **lg (平板横屏)**：宽度 ≥ 840px，采用多栏横向布局

适配策略：

- 使用 `@Builder` 装饰器封装不同布局
- 通过 `windowWidth` 动态判断设备类型

- 响应式字体大小和间距调整

7.2 传感器集成技术分析

光线传感器应用场景：

- 自动亮度调节：根据环境光线调整屏幕亮度
- 环境监测：记录光照变化数据

陀螺仪传感器应用场景：

- 运动检测：检测设备姿态变化
- 游戏交互：实现体感控制
- 导航辅助：辅助定位方向

数据采集准确性：

- 光线传感器精度：0-100000 lux
- 陀螺仪精度：0.01 rad/s
- 采样频率：10Hz

7.3 分布式能力分析

分布式数据管理原理：

```
import distributedData from '@ohos.data.distributedData';

const KVManager = distributedData.createKVManager({
  bundleName: 'com.example.weather',
  kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

async function syncWeatherData(key: string, value: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  await kvStore.put(key, value);
}
```

跨设备同步实现：

- 基于HarmonyOS分布式软总线
- 支持多设备数据实时同步
- 自动处理网络状态变化

八、总结与思考

8.1 实验总结

通过本实验，我掌握了以下核心技能：

1. **ArkUI声明式开发**：学会使用声明式语法构建UI界面

2. **多端响应式布局**：掌握断点系统和响应式设计
3. **传感器数据采集**：理解传感器API调用流程
4. **分布式能力**：了解鸿蒙分布式数据管理原理

8.2 课后思考

1. 鸿蒙多端开发相比其他框架的优势：

- 一套代码多端部署
- 原生性能体验
- 分布式能力原生支持

2. 传感器在物联网中的应用场景：

- 环境监测系统
- 智能家居控制
- 健康监测设备

3. 分布式技术对未来移动开发的影响：

- 打破设备边界
- 实现多设备协同
- 构建全场景智慧生活

实验日期：2026年5月9日

实验者：王永喜